PRODUCTION DASHBOARD

Support and Maintenance

Costello Windows production dashboard

Version of 08 September 2026 · build 20260908-1849 · source: docs/SUPPORT-AND-MAINTENANCE.md

Contents

This document is for whoever maintains the Costello Windows production dashboard: what it is built from, how it fits together, where every piece of data lives, how to test and deploy it, how to carry out the routine operations, and how to fix the problems that come up. It assumes the reader is comfortable with a browser's developer tools, SharePoint, and running a command line — it is not a guide to using the dashboard day to day (see *User Guide* for that).

---

1. Overview and the two pages

The dashboard is a static site — no build step, no framework, no bundler — of plain browser JavaScript, served by GitHub Pages directly from the repository's `main` branch. It reads and writes one shared SharePoint workbook (its `Production` sheet) through the Microsoft Graph Excel API, plus several SharePoint lists for state that must never touch the workbook.

Two pages ship from the one repository:

- **`index.html` + `app.js`** — the Master dashboard, used by the office.
- **`glass.html` + `station.js` + `station-core.js`** — the Glass station, used on a shared tablet on the glass floor, signed in with a shared account that has no access to the workbook at all.

---

2. Components

Component	Version	Used for
Microsoft Entra app registration (public client, delegated)	—	The dashboard's own app identity in Entra ID. Delegated permissions only — the app never acts as itself, only as whoever is signed in.
MSAL browser	v3.28.1 (`vendor/msal-browser.min.js`)	Sign-in (`loginPopup`), silent token acquisition, and the scope split between workbook and SharePoint-list access.
Microsoft Graph v1.0	—	Excel workbook API (download, session, surgical writes), drive item lookups, SharePoint list reads/writes including delta queries.
ExcelJS	4.4.0 (`package-lock.json` ; the vendored browser build is dated 19-10-2023)	In-browser: parses the downloaded workbook (`vendor/exceljs.min.js`). In Node: builds in-memory workbooks for the offline tests and for the Export-to-Excel feature.
pdfmake	v0.2.23 (`vendor/pdfmake.min.js` , lazy-loaded)	Building the PDF export, plus its font data (`vendor/vfs_fonts.js`).
GitHub Pages	—	Static hosting straight from `main` ; a push is the deploy.

Component	Version	Used for
SharePoint	—	Hosts the one production workbook, the dashboard's own sheets on it, and the SharePoint lists described in §5.
Node.js	—	Runs the offline test suites (`test_*.js`) and the alerts digest tooling in `automation/` .
Python	—	Runs `build.py` , the build-stamping script.
Google Fonts	Barlow Condensed (weights 600, 700); IBM Plex Sans (weights 400, 500, 600, 700)	Loaded via a `<link>` to `fonts.googleapis.com` on both pages; the only external network dependency besides Microsoft Graph itself.

3. Architecture

3.1 Three flows, in text

Office read/write (workbook):

```
SharePoint site (workbook's own site)
  -> sign in (MSAL, Entra ID)                          graph.js
  -> findFile(): locate the workbook by name            graph.js
  -> downloadWorkbook(): GET the whole .xlsx             graph.js
  -> ExcelJS workbook in memory (values, fills, formats)
  -> parseWorkbook(wb)                                  parser.js
  -> job model: one plain object per job
  -> applyPending() overlays this browser's own
     unconfirmed writes                                 app.js
  -> rendered: job list / drawer / phase pipeline / export window
  -> a tap or a form submit
  -> a surgical write through the Excel API:
       setFill() on one cell (checkpoints)               graph.js
       moveJobRow() (the only structural write)          graph.js
       upserts to Dashboard Log/Views/Progress/Alerts     graph.js
       upserts to the Dashboard phases list (never the
       workbook itself)                                  graph.js
```

Office feeds the floor:

```
app.js load() succeeds
  -> feedStation(): builds the glass slice of "in
     production" jobs, one row per job, Total = DG + TG   app.js
  -> diffs against the "Glass station" list                 app.js
  -> sends adds / fact patches / seeded counters
     (only on a new row or one never tapped)                graph.js -> Glass station list
                                                                        (Floor stations site)
  -> tablet's next poll picks the change up
```

Floor records progress:

```
a tap on a card (Cutting / Hotmelting / Glazing, +/-/All)  station.js
  -> queued locally (cw_stationq), whitelisted to the
     floor's own fields                                    station-core.js
  -> PATCH the counter + its By/At + the overall
     DoneBy/DoneAt                                          graph.js -> Glass station list
  -> one "Station log" line queued and POSTed                graph.js -> Station log list
  -> office picks both up on its next delta poll (<=10s
     while a glass screen is open)                          app.js (listDelta)
```

3.2 The scope split

`graph.js` defines two scope sets:

- `SCOPES` (`Files.ReadWrite.All`, `User.Read`) — used for sign-in and every workbook call. Every dashboard user has these the moment they sign in.
- `LIST_SCOPES` (`Files.ReadWrite.All`, `Sites.ReadWrite.All`, `User.Read`) — needed only for SharePoint list requests (`/lists` paths, and the by-path lookup for the floor's separate site). Requested quietly in the background so a tenant that has not yet granted `Sites.ReadWrite.All` still gets a fully working workbook dashboard; a popup is shown only from a genuine click. `needsListScope(path)` decides which scope set to use based on the *shape* of the request path, never on characters that might appear inside an A1 range address (an earlier bug: a colon test for "is this a list path" matched every A1 range and broke every workbook write).

This split exists so a missing SharePoint permission degrades gracefully — phases and the glass station simply report that they need permission, rather than sign-in itself failing for everyone.

3.3 The workbook session and the 36-second file lag

SharePoint takes roughly 35 seconds to fold a change made through the Excel API into the file `downloadWorkbook()` would fetch; the Excel API itself reflects the same change in about a second. To stop a person's own edit appearing to work and then vanishing on the next refresh, every optimistic write is held locally (`PENDING`, in memory and `localStorage` as `cw_pending`) for up to 180 seconds per field, with its own timestamp. `applyPending()` overlays these holds onto whatever was just parsed, and drops a hold early the moment a freshly downloaded file agrees with it. The same pattern covers section moves (`cw_pendv`), alerts (`cw_penda`) and hand-set phases.

3.4 The five sections and row moves

Jobs sit in the sheet's five sections (divider rows, never a job row). Moving a job between sections moves its **row** in Excel: `moveJobRow()` locates the job live, captures its values and number formats in one read (fills and fonts come from the download), inserts a row after the target section's last job, writes it (guarding any text that looks like a number, formula, boolean or date fragment with a leading apostrophe), verifies exactly two copies exist, then deletes the original.

Border rule. Excel keeps one border definition per shared edge between two rows, so the landing row writes its bottom edge and verticals but never its top, and the row above a landing or vacated slot has its bottom edge explicitly re-asserted afterwards (`restoreBottomEdge`) — the shared line "belongs" to whichever row did not move.

3.5 Checkpoints and colours

A checkpoint tick writes only a colour to the item's own cell on `Production` — white (nothing), yellow (part done), gold (done). The exact count behind that colour lives in the `Dashboard Progress` sheet, plus one line in `Dashboard Log` . On refresh, the sheet's own colour always wins over a stored count.

3.6 The phase list

The seven-step phase pipeline is derived automatically from what is already on the sheet (`jobPhase()` in `checkpoints.js`); clicking a step by hand stores an override in the `Dashboard phases` SharePoint list, never in the workbook. `effectivePhase = max(sheet-derived, hand-set)` .

3.7 The station site lookup and the interim fallback

`stationSite()` prefers the separate `Floor stations` site; while that site does not exist (creating one needs a Global Administrator the owner does not currently have), the same three lists live in the workbook's own site and both pages fall back to it, re-checking for the real site every ten minutes and switching over on their own the moment it appears — dropping any cached delta tokens or list ids from the old site so nothing is misapplied. In this interim arrangement the station account is a member of the workbook's own site and can therefore open the workbook if pointed at `index.html` ; the owner has accepted that trade-off until the separate site can be created.

3.8 Delta polling and the generation counter

Both pages keep their SharePoint lists current with `listDelta()` rather than re-reading the whole list each time: the first call enumerates and returns a `deltaLink` ; each call after passes that token back and gets only what changed. A 410 (the delta token has gone stale) is answered by reading the list once and starting a fresh enumeration; a list that refuses delta outright is polled the plain way for five minutes before being retried. A generation counter discards any delta result that arrives after the list it answers for has since been replaced by a full resync, so a stale reply can never be applied on top of newer data.

3.9 The tablet's queue

Every tap is applied to the screen immediately, queued in `cw_stationq` (whitelisted to the floor's own fields), and drained until empty — the queue is never snapshotted and then dropped, because a tap landing mid-write would otherwise be lost. It retries on failure, survives a reload, and a replayed entry is re-whitelisted to the counter fields on the way back out. Once a counter write lands, exactly one `Station log` line is queued (`cw_stationlogq`) and sent — never before, and never if the counter write itself never happened. A queued entry whose row has since risen under it (a seed arriving while the tablet was offline) is re-based rather than overwriting the seed.

4. File map

File	Purpose
`index.html`	Master dashboard: page shell, styles, script tags
`glass.html`	Glass station page shell and styles
`graph.js`	Entra sign-in (MSAL) and every Microsoft Graph read/write call
`parser.js`	Workbook → job model; runs in the browser and in Node
`checkpoints.js`	Checkpoint merge rule, colours, debounced writes, the phase pipeline
`export.js`	Pure export logic: filtering, rows, Excel workbook, PDF document definition
`station-core.js`	Pure glass-station logic (slice/plan/board/tap/people/log) — no DOM, no Graph
`station.js`	Glass station page UI
`app.js`	Master dashboard UI: rendering, drawer, windows, wiring
`build.py`	Stamps a build id (`?v=…`) onto every script tag on both pages, writes `version.json`
`verify.js`	Dev-only check: compares the JS parser against a Python reference extract
`rehearse.js`	Harness a scratch script drives to run the real `graph.js` against a live token and workbook reference (neither of which is in the repo)
`automation/`	The alerts mailer: Office Script source, its generator, `SETUP.md`
`vendor/`	Vendored MSAL, ExcelJS and pdfmake plus pdfmake's font data — no CDN dependency at runtime
`test_*.js` , `automation/test_digest.js`	Offline test suites, one per feature
`docs/`	Architecture, support, stations, specs index, and this document
`package.json` / `package-lock.json`	Node dependency (`exceljs`) used by the offline tests and the Export-to-Excel builder

Scratch and rehearsal tooling (outside the repository)

Kept in the session's scratchpad directory, never checked in:

Tool	Purpose
`graph.py` , `token.json` , `fileref.json`	A device-code token and workbook reference for one-off Graph scripts
`rehearse_*.js` , `live_*.js`	Run the real `graph.js` against a OneDrive copy of the workbook, then one net-zero check on the live file
`fidelity.py` , `cp_fidelity.py` , `com_check.py`	Compare a rehearsed workbook against the original, including drawn borders via desktop Excel COM

Tool	Purpose
`make_phase_list.py`, `make_station_lists.py`	Create/complete the SharePoint lists with the exact columns (idempotent; has a `check` mode)
`stub_station.js`, `cors_server.py`	A fake sign-in library and fake Graph, injected with Playwright, so both pages run offline against a fixture for screenshots and behaviour checks

5. Data and state

5.1 Workbook sheets the dashboard owns

Sheet	Constant	Written by	Columns / content
`Dashboard Log`	`LOG_SHEET`	`appendLog()`	One line per change anywhere in the app
`Dashboard Views`	`VIEWS_SHEET`	`saveAssignment()` / `clearAssignment()`	Custom groupings, per job
`Dashboard Progress`	`PROGRESS_SHEET`	`saveProgress()` / `saveProgressMany()`	Job | Item | Done | Total | Who | When — exact checkpoint counts
`Dashboard Alerts`	`ALERTS_SHEET`	`addAlert()` / `removeAlert()`	Job | Email | Added by | When — subscriptions
`Dashboard Config`	`CONFIG_SHEET`	**read-only** — never created or written by code	Key | Value — the admin address, the dashboard's own URL, and similar configuration

5.2 SharePoint lists

List	Site	Written by	Columns
`Dashboard phases`	the workbook's own site	office (dashboard)	Title (job, unique), Phase, PhaseName, SetBy, SetAt
`Glass station`	`Floor stations` (or the interim fallback)	feeder (job facts + seeded counters); tablet (counters, By/At, last-touch)	Title, Job, Customer, GlassType (`GLASS`), Total, Seq, Active, FedAt, FedBy — feeder; Cut, Hotmelt, Glazed, CutBy/At, HotmeltBy/At, GlazedBy/At, DoneBy, DoneAt — tablet, or the feeder only while seeding
`Station people`	`Floor stations`	admin, by hand in SharePoint	Title, Station, Stages, PIN, Active
`Station log`	`Floor stations`	tablet only	Title (job), Station, GlassType, Stage, From, To, Who, At

5.3 `localStorage` keys per page

Both pages / shared logic:

Key	Holds
`cw_fileref`	The resolved workbook file reference
`cw_listids`	Resolved SharePoint list ids, once found
`cw_stationsite`	Which site (`Floor stations` or the interim fallback) is currently in use

Master dashboard (`index.html` / `app.js`) only:

Key	Holds
`cw_pending`	Unconfirmed checkpoint/phase writes, per field, with timestamps (`PENDING_MS` = 180 s)
`cw_pendv`	Unconfirmed view/section assignments
`cw_penda`	Unconfirmed alert subscriptions
`cw_cpqueue`	A queued/not-yet-sent checkpoint tap, replayed on next load
`cw_stationfeed`	When `feedStation()` last ran, and the hash of what it sent
`cw_exportpresets`	Saved export filters/fields/format — never job data
`cw_theme` , `cw_hidden` , `cw_collapsed` , `cw_changes`	UI-only preferences

Glass station tablet (`glass.html` / `station.js`) only:

Key	Holds
`cw_stationq`	Queued, not-yet-sent counter taps
`cw_stationlogq`	Queued, not-yet-sent `Station log` lines
`cw_person`	Who is currently signed in on the tablet, and the time of their last tap — never which stages they hold (always re-read from the list)
`cw_stationtheme`	The tablet's own light/dark preference, independent of the office's

---

6. The hard rules, and how the tests prove them

From `CLAUDE.md` :

Rule	Proved by
`Production` is only ever changed by sanctioned single-cell colour writes or a whole-row section move	`test_move.js` (row-move protocol), `test_checkpoints.js` (colour-only writes)
No other sheet is touched except the dashboard's own, and `Dashboard Config` is read-only	Every suite's fake `fetch` asserts requests only against known sheet names; `test_alerts.js` and others assert `Dashboard Config` is never PATCHed

Rule	Proved by
The office never writes `Station people` or `Station log`, and never writes the floor's own `Glass station` columns except the one seeding exception	`test_station.js` — asserts every feeder write is a subset of `ST.FEEDER_WRITES`, and no request from the office path touches the floor-only fields outside that exception
No phone number or eircode ever leaves the app in an export	`export.js` 's standing test in `test_export.js`
Every export is logged once	`test_export.js`
No real name/address/domain in the repository	Manual review; enforced by convention, not a test

`test_station.js` alone runs 158 checks proving, over the whole run: no request touched the workbook, no `DELETE` was ever sent, every floor `PATCH` is a subset of the floor's own columns, every feeder write is inside `ST.FEEDER_WRITES`, and no request body ever carried a phone number, eircode, county, price or comment.

---

7. Testing, building, deploying

7.1 Commands

```
set -o pipefail
node --check parser.js && node --check graph.js && node --check checkpoints.js \
  && node --check export.js && node --check app.js \
  && node --check station-core.js && node --check station.js

node test_move.js
node test_checkpoints.js
node test_alerts.js
node test_export.js
node test_phases.js
node test_phases_list.js
node automation/test_digest.js
node test_station.js

node verify.js   # dev-only cross-check, needs local files not in the repo
```

Always run with `set -o pipefail` — a failing test earlier in a pipeline must not be masked by a later command that succeeds.

7.2 The offline test pattern

Every `test_*.js` file (and `automation/test_digest.js`) runs in plain Node, with no network and no real Graph/SharePoint/Excel connection:

- The real source files are loaded with `vm.runInThisContext(fs.readFileSync(...), { filename: ... })` — the actual shipped code runs, not a re-implementation of it.

- `global.window`, `global.localStorage` and a stub DOM element factory stand in for the browser; `global.ExcelJS` is either a tiny fake or the real `exceljs` package.
- `global.fetch` is a hand-written fake that inspects the method, URL and body of every call against an in-memory fixture and returns canned Graph-shaped JSON. Several suites assert `fetch` was *never* called down certain paths — that is how "no network" and "workbook never touched" are proved, not merely commented.

7.3 What each suite covers

Suite	Covers	Checks
`verify.js`	Parser vs a Python reference extract (local files only)	agree/disagree
`test_move.js`	Row-move protocol	7
`test_checkpoints.js`	Checkpoint logic and writes	36
`test_alerts.js`	Subscriptions, admin gate, pending holds	30
`test_export.js`	Builders, no phone/eircode, logging	45
`test_phases.js`	Phase derivation and the wheel	26
`test_phases_list.js`	The phases list, scope split, dedupe	35
`test_station.js`	Floor stations end to end (offline)	158
`automation/test_digest.js`	The alerts digest	26

7.4 `build.py`

```
python build.py
```

Stamps a `?v=<timestamp>` (format `YYYYMMDD-HHMM`) onto every `<script src="…">` tag matching `parser`, `graph`, `checkpoints`, `station-core`, `station`, `export` or `app`, on **both** `index.html` and `glass.html`, updates the `<span id="build">` footer text on each, and writes `version.json` with the same build id. Adding a new script tag to either page that should be cache-busted this way means extending the `SCRIPTS` regex or the `stamp()` calls at the bottom of `build.py`.

7.5 GitHub Pages and build freshness

Deployment is: run every check green, `python build.py`, commit (only once the owner has approved — see §8's alerts flow and the manager loop in `CLAUDE.md`), push to `main`. GitHub Pages serves `main` directly — there is no separate deploy step. GitHub Pages can cache a page for up to about ten minutes, so a browser can sit on a stale build right after a push.

Both pages notice a new build on their own:

- `index.html` / `app.js`: `checkBuild()` polls `version.json` every two minutes and shows a "A newer version of the dashboard is available — Reload" banner once the server is ahead of what is loaded.
- `glass.html` / `station.js`: polls the same way and reloads itself automatically once the build has moved on — but only when its own write queue is empty, so a tablet mid-tap is never interrupted.

8. Operations

8.1 Adding a person or PIN

Add a row to `Station people` in SharePoint: `Title` = the name shown on the tablet, `Station` = `Glass`, `Stages` = a comma-, semicolon- or slash-separated list of `cut`, `hotmelt`, `glazed` (anything else is silently ignored), `PIN` = 4–6 digits or blank for no PIN, `Active` = `Yes`. The tablet re-reads this list every ten minutes on its own, and also at start-up — no sign-out or reload needed for a new row, a stage change, or `Active` flipped to `No` to retire someone.

8.2 Adding a future station

1. Extend the `STATIONS` array in `app.js` (currently `[["glass", "Glass station"]]`) with the new station's key and label — this adds it to the Master dashboard's **Show** dropdown.
2. Create a new SharePoint list for it, shaped like `Glass station`: feeder-owned fact columns, station-owned progress columns, a unique `Title`.
3. Copy `glass.html` (and `station.js`, adapted) into a new page for the new area, pointed at the new list.

Each station's data stays isolated in its own list and its own page; a station account for one area never needs, and never gets, access to another's data.

8.3 Moving the lists to the separate Floor stations site later

While `Floor stations` cannot yet be created (no Global Administrator available), the three floor lists live in the workbook's own site. Moving them later needs an administrator and four steps, no deploy:

1. Create the private team site `Floor stations`; add the admin as owner and the station account as a member.
2. Run the list-creation script against the new site.
3. Copy the rows across from the three lists in the workbook's site.
4. Remove the station account from the workbook's own site.

Both pages pick up the new site within ten minutes on their own, or immediately on their next reload — nothing in the repository changes. A tap queued before the move is not lost: each queued entry is stamped with the site it was made in, and after a move the row is re-found by its Title (the job number). A tap whose row did not come across at all is dropped with a console message rather than written to whatever row happens to share that item id in the new list.

8.4 Clearing test rows

Test or rehearsal rows created in `Dashboard phases`, `Glass station`, `Station people` or `Station log` during development should be deleted by hand in SharePoint before real use — the dashboard has no bulk-delete feature for any of them by design (nothing in the app deletes list items).

8.5 Changing the admin address

Edit the `admin` row's Value on the `Dashboard Config` sheet directly in Excel (the dashboard only ever reads this sheet, never writes it). This address controls who may add or remove alert subscriptions, and which domain alert recipients must be on.

8.6 The alerts flow

The dashboard only manages *subscriptions* (`Dashboard Alerts` sheet); the mail itself is sent by a Power Automate flow running an Office Script, generated from `automation/digest-core.js` via `automation/build-script.js` into `automation/alerts-digest.ts`. Setting this up is a one-off, click-by-click job for the admin — see `automation/SETUP.md` in full, including its own troubleshooting table. In short: paste the generated script into Excel Online's Automate tab, build a Power Automate scheduled flow (every 3 days at 08:00 Dublin time) that runs the script, parses the JSON it returns, and sends one email per address. Never edit the `.ts` file directly — regenerate it from the `.js` core and re-paste.

---

9. Troubleshooting

Symptom	Cause	Fix
Sign-in won't complete	Account has no access to the `ProductionProgress` site, or a popup was blocked	Confirm the account can already open the workbook in Excel; check the browser isn't blocking `loginPopup`; a hard refresh usually clears a stuck redirect
"Permission needed" for phases or the glass station	Signed-in account has workbook scopes but not `Sites.ReadWrite.All` yet, and it has never been granted tenant-wide	Entra admin centre → App registrations → the app → API permissions → confirm/add `Sites.ReadWrite.All` (delegated) → **Grant admin consent** for the organisation. One-off, tenant-wide.
"List not found"	A required list is missing, in the wrong site, or has a mistyped column	Check the list exists with the exact name in the right site; check columns match the spec exactly (columns are read by name, never repaired automatically); list ids are cached only once found, so creating the list and reloading (or "Try again" on the tablet) is enough
Workbook locked (423)	Someone has it open in desktop Excel without AutoSave on	Ask them to close it or enable AutoSave
`InvalidSession`	The workbook session went idle and Graph expired it	Retried automatically (fresh session, one resend); if persistent, the workbook is likely under heavy load — wait and retry
`429` / `OperationQueueFull`	Excel's API queue was flooded (bulk move, big batch of checkpoint writes)	Retried automatically with backoff; if it still fails, several people are probably writing at once — space bulk actions out

Symptom	Cause	Fix
Stale build showing after a deploy	GitHub Pages caches the page for up to ~10 minutes	Check the footer build number; wait for the reload banner, or hard-refresh (Ctrl+Shift+R / Cmd+Shift+R)
Alerts not sending	The mail flow itself (not the dashboard) has an issue	See `automation/SETUP.md`'s own troubleshooting table — nothing about the mail flow can be debugged from the dashboard
Station tablet says a site or list is missing	Setup in `docs/STATIONS.md` is not yet finished	Walk through the setup steps, then tap "Try again" (also clears the cached site id)
Station tablet: "Tap Sign out, then Sign in again"	The station account's sign-in itself has expired (not a permission issue)	Sign out, then sign in again with the station account; queued taps and log lines are kept and sent afterwards
A person is missing from the tablet's picker	Their `Station people` row is missing, `Station` isn't exactly `Glass`, or `Active` isn't exactly `Yes`	Fix the row in SharePoint; the tablet re-reads it within ten minutes, or on reload
Their steppers are greyed for a stage they should hold	`Stages` doesn't include a recognised key	Only `cut`, `hotmelt`, `glazed` are recognised (comma/semicolon/slash separated, case-insensitive); anything else is silently dropped
PIN not accepted	Typo, stray space, or wrong digits in the `PIN` column	Check the column directly — there is no server-side check or lockout, so a wrong PIN just shakes and says "Try again"
A job is missing from the floor's board	It isn't "in production" yet/any more, or hasn't been fed yet	Only "in production" jobs are fed; the feed only runs when an office dashboard is open and successfully loads, and skips a run if nothing changed in the last ten minutes
"cannot reach SharePoint — retrying"	A read that worked before has failed for now (network, rate limit, gateway)	Nothing to click — the last good board/drawer stays on screen and it keeps retrying on its own clock; check tenant SharePoint/Graph status if it doesn't clear in a few minutes
The station account can open the master page	Expected in the interim site arrangement (§3.7) — otherwise a real problem	If the interim arrangement has already been superseded by the separate site, check the station account's site memberships — it should have none on the workbook's site
The Floor log window shows nothing for an old job	The 90-day display window (`LOG_DAYS`) has passed	Nothing is deleted from `Station log` itself — this is a display limit, not data loss
Need to roll back a bad workbook edit	A wrong colour or section move happened	Open **Versions** in the Master dashboard, review the diff, restore the version from before the mistake — the restore itself becomes a new, rollable version
Need to roll back a bad dashboard release	A deploy introduced a regression	`git revert` the offending commit, run `python build.py`, push to `main` — there is no separate rollback mechanism; the site is always whatever `main` currently is

10. Known limits and honest caveats

- **No push from Graph.** Microsoft Graph has no real-time push for either the workbook or SharePoint lists; everything here is polling — 10-second delta polling on the tablet, and on the office side while a glass screen is open, dropping back to once a minute otherwise.
- **PIN and gating are deterrents, not security.** The PIN is compared on the tablet itself against a column the station account can read; the ten-minute lock and stage gating are enforced the same way. None of it is authentication — the real boundary is that the station account can reach only the `Floor stations` site (or its interim fallback) and nothing else.
- **The feeder needs an office dashboard open.** A job only ever reaches the floor because a Master dashboard, open in someone's browser, successfully loaded the workbook and ran `feedStation()`. If nobody has the office page open, the floor's board simply goes stale until someone does.
- **The interim site arrangement exposes the workbook to the station account.** Until a separate `Floor stations` site can be created (needs a Global Administrator not currently available), the floor's three lists live in the workbook's own site and the station account is a member of that site — meaning it technically could open the workbook if pointed at `index.html`. The owner has accepted this trade-off for now; see §3.7 and §8.3 for the planned fix.
- **GitHub Pages cache of up to ten minutes.** A push can take a few minutes to actually reach browsers; the footer build number and the reload banner are the way to confirm a deploy has landed.
- **iOS zoom.** Form inputs on the tablet are kept at 16px font size specifically to stop iOS Safari auto-zooming into a field on focus.
- **90-day log horizon.** The Floor log window and a job's drawer timeline only display the last 90 days of `Station log` lines (`LOG_DAYS` in `station-core.js`); nothing is ever deleted from the list itself, so older lines still exist in SharePoint even though the dashboard stops showing them.